

Usando *decorator*

Um **decorador** é uma função que recebe outra função e devolve uma versão "embrulhada", com algo a mais antes ou depois: registrar chamadas, medir tempo, validar argumentos, guardar resultados. O `@` em cima do `def` é só um atalho para isso. Nesta aula: funções como objetos, closures, o decorador feito à mão e com `@`, `functools.wraps`, decoradores com argumentos e o `@cache` da biblioteca padrão.

Função é objeto

Closure

@decorador

*args · **kwargs

functools.wraps

Com argumentos

@cache

Formação Claude Code 2026 · Baseado em docs.python.org (Glossário: decorator; Reference: Function definitions; functools: wraps, cache; time.perf_counter) e PEP 318, consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Funções são *objetos*

objetos.py

```
def saudar(nome):  
    return f"Olá, {nome}!"  
  
falar = saudar                # a função é um objeto  
print(falar("Ana"))  
  
def aplicar(funcao, valor):    # recebida como argumento  
    return funcao(valor)  
  
print(aplicar(len, "decorador"))  
  
def criar_multiplicador(fator):  
    def multiplicar(n):        # função criada dentro de outra  
        return n * fator      # e lembra de 'fator' (closure)  
    return multiplicar  
  
triplo = criar_multiplicador(3)  
print(triplo(7))
```

SAÍDA

```
Olá, Ana!  
9  
21
```

- ✓ Uma função pode ir para outra variável, sem parênteses
- ✓ Pode ser passada como argumento (como no `map` e no `filter`)
- ✓ Pode ser criada dentro de outra e devolvida
- ✓ A função interna lembra as variáveis de fora (`fator`): isso é uma **closure**

Essas três ideias juntas são tudo de que um decorador precisa.

Um decorador *sem o @*

manual.py

```
def com_moldura(funcao):  
    def embrulho():  
        print("=" * 12)  
        funcao()           # chama a função original  
        print("=" * 12)  
    return embrulho  
  
def boas_vindas():  
    print(" Bem-vindo!")  
  
boas_vindas = com_moldura(boas_vindas)  # decorando à mão  
boas_vindas()
```

SAÍDA

```
=====  
 Bem-vindo!  
=====
```

RECEBE
a função original



CRIA
o embrulho



DEVOLVE
o embrulho

- ✓ `com_moldura` não chama nada: só monta e devolve `embrulho`
- ✓ `boas_vindas = com_moldura(boas_vindas)` troca a função pela versão embrulhada
- ✓ Quem chama `boas_vindas()` ganha as molduras sem saber

O atalho @ e os argumentos

arroba.py

```
def registrar(funcao):
    def embrulho(*args, **kwargs):
        print(f"-> {funcao.__name__}{args}")
        resultado = funcao(*args, **kwargs)
        print(f"<- {resultado}")
        return resultado
    return embrulho

@registrar          # o mesmo que somar = registrar(somar)
def somar(a, b):
    return a + b

total = somar(2, 3)
print("total:", total)
```

SAÍDA

```
-> somar(2, 3)
<- 5
total: 5
```

- ✓ @registrar em cima do def faz somar = registrar(somar)
- ✓ O embrulho recebe *args, **kwargs para aceitar qualquer assinatura (aula 20 do módulo 37)
- ✓ Devolva o resultado da função original: sem o return, quem chama recebe None

Sem perder *o nome e a docstring*

usando_wraps.py

```
from functools import wraps

def sem_wraps(funcao):
    def embrulho(*args, **kwargs):
        return funcao(*args, **kwargs)
    return embrulho

def com_wraps(funcao):
    @wraps(funcao)          # copia nome, docstring...
    def embrulho(*args, **kwargs):
        return funcao(*args, **kwargs)
    return embrulho

@sem_wraps
def area(l):
    """Área de um quadrado."""
    return l * l

@com_wraps
def perimetro(l):
    """Perímetro de um quadrado."""
    return 4 * l

print(area.__name__, "|", area.__doc__)
print(perimetro.__name__, "|", perimetro.__doc__)
```

SAÍDA

```
embrulho | None
perimetro | Perímetro de um quadrado.
```

- ✓ Sem `wraps`, a função decorada passa a se chamar `embrulho` e perde a docstring
- ✓ Isso confunde mensagens de erro, logs, `help()` e ferramentas de teste
- ✓ `@wraps(funcao)` no embrulho copia esses dados da original

Regra prática: todo decorador seu leva `@wraps`.

Um *cronômetro* para qualquer função

cronometro.py

```
import time
from functools import wraps

def cronometrar(funcao):
    @wraps(funcao)
    def embrulho(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = funcao(*args, **kwargs)
        gasto = time.perf_counter() - inicio
        print(f"{funcao.__name__} levou {gasto:.1f} s")
        return resultado
    return embrulho

@cronometrar
def relatorio():
    time.sleep(0.3)          # simula um trabalho demorado
    return "pronto"

print(relatorio())
```

SAÍDA

```
relatorio levou 0.3 s
pronto
```

- ✓ O código de medir tempo fica num lugar só
- ✓ Basta pôr `@cronometrar` em qualquer função para medir
- ✓ `time.perf_counter()` é o relógio certo para medir intervalos

O `sleep(0.3)` só simula uma tarefa lenta; o tempo medido varia um pouco a cada execução.

@repetir(3): *um nível a mais*

com_argumentos.py

```
from functools import wraps

def repetir(vezes):
    # recebe o argumento...
    def decorador(funcao):
        # ...e devolve o decorador
        @wraps(funcao)
        def embrulho(*args, **kwargs):
            for _ in range(vezes):
                resultado = funcao(*args, **kwargs)
            return resultado
        return embrulho
    return decorador

@repetir(3)
def avisar(msg):
    print("aviso:", msg)

avisar("backup concluído")
```

SAÍDA

```
aviso: backup concluído
aviso: backup concluído
aviso: backup concluído
```

REPETIR(3)

recebe o
argumento



DECORADOR

recebe a função



EMBRULHO

roda na chamada

- ✓ `@repetir(3)` primeiro chama `repetir(3)`, que devolve o decorador de verdade
- ✓ Por isso são três funções, uma dentro da outra
- ✓ O `vezes` fica guardado pela closure

@cache: *lembrar resultados*

usando_cache.py

```
from functools import cache

chamadas = 0

@cache
def fib(n):
    global chamadas
    chamadas += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(40), "em", chamadas, "chamadas")
print(fib(40), "de novo: ainda", chamadas)
```

SAÍDA

```
102334155 em 41 chamadas
102334155 de novo: ainda 41
```

- ✓ `functools.cache` guarda o resultado de cada argumento já calculado
- ✓ Fibonacci recursivo (aula 19 do módulo 37) cai de centenas de milhões de chamadas para 41
- ✓ Na segunda vez, nem uma chamada nova: o resultado vem da memória

Use só em funções que, para os mesmos argumentos, sempre devolvem o mesmo resultado e não têm efeitos colaterais.

Só números *positivos*

Enunciado: crie o decorador `@exige_positivos`, que recusa com `ValueError` qualquer chamada com número menor ou igual a zero.

Uma resposta possível

```
from functools import wraps

def exige_positivos(funcao):
    @wraps(funcao)
    def embrulho(*numeros):
        if any(n <= 0 for n in numeros):
            raise ValueError(f"{funcao.__name__}: só positivos")
        return funcao(*numeros)
    return embrulho

@exige_positivos
def area_retangulo(base, altura):
    return base * altura

print(area_retangulo(3, 4))
try:
    area_retangulo(3, -4)
except ValueError as erro:
    print("Erro:", erro)
```

SAÍDA

```
12
Erro: area_retangulo: só positivos
```

- ✓ A validação (aula 07) sai de dentro da função e vira reaproveitável
- ✓ Graças ao `@wraps`, a mensagem mostra o nome certo: `area_retangulo`
- ✓ Desafio: aceite também argumentos nomeados (`**kwargs`)

Teste o que *ficou*

1. @log em cima de def f(): ... equivale a:

A) $f = \log$

B) $f = \log(f)$

C) $\log = f(\log)$

D) $f()$

B.

2. O embrulho esqueceu o return. A função decorada devolve:

A) O resultado normal

B) None

C) O embrulho

D) Erro

B.

3. Para que serve o @wraps(funcao)?

A) Deixar mais rápido

B) Manter nome e docstring da original

C) Aceitar argumentos

D) Guardar resultados

B.

4. Um decorador com argumento, como @repetir(3), tem quantos níveis de função?

A) 1

B) 2

C) 3

D) 4

C. repetir, decorador e embrulho.

Algo a mais, *sem mexer na função*

Um decorador recebe uma função, cria um embrulho com `*args, **kwargs` que chama a original e devolve esse embrulho; o `@` faz a troca para você. Use `@wraps` sempre, devolva o resultado, acrescente um nível quando o decorador tiver argumentos e aproveite os prontos, como o `@cache`.

```
f = deco(f)
```

```
*args · **kwargs
```

```
@wraps
```

```
@deco(arg)
```

```
@cache
```

Próxima aula → 11 · Programação Assíncrona

Fontes: docs.python.org/3/glossary.html#term-decorator; docs.python.org/3/reference/compound_stmts.html#function-definitions; docs.python.org/3/library/functools.html (`wraps`, `cache`); docs.python.org/3/library/time.html#time.perf_counter; peps.python.org/pep-0318, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.